Bug Hunt v2-1: Text Analysis

What the program does

This problem consists of **two modules** plus a driver:

- `tokenizer.erl` — turns a raw string into a list of normalized words. Exports: `tokenize/1` (the pipeline), `lowercase/1`, `strip_punctuation/1`, `split_words/1`.
- `analyzer.erl` — operates on a list of words. Exports: `word_freq/1` (count occurrences), `longest_word/1`, `palindromes/1` (words that read the same forwards and backwards, length 2+ only), `most_frequent/2` (the N most-occurring words), `least_frequent/2` (the N least-occurring).
- `main.erl` — runs a fixed paragraph of English through the pipeline, prints the tokens, the word-frequency table, a few summary analyses, and a self-consistency check on the total token count.

The intended tokenization rules:

- Uppercase letters are converted to lowercase.
- ASCII punctuation (`.,;:!?"'()[]{}-`) is replaced by spaces (so that a word followed immediately by punctuation gets correctly separated from anything that comes after).
- The resulting string is split on whitespace, with empty tokens dropped.

The intended analyzer behavior:

- `word_freq` returns `[{Word, Count}]` in first-seen order. A word that appears `K` times has count `K`.
- `longest_word` returns the longest word; ties broken by first occurrence.
- `palindromes` returns words that equal their reverse, filtering out length-1 words.
- `most_frequent(Words, N)` returns the N pairs with the highest counts.
- `least_frequent(Words, N)` returns the N pairs with the lowest counts.

The catch

This program contains **exactly 4 bugs** that prevent it from producing the expected output. The bugs are all *semantic* — the program compiles without errors and runs to completion without crashing.

This problem is harder than the round-1 bug hunts in four specific ways:

1. **At least one bug's cause lives in a different file from where its symptom appears.** Following the symptom directly will land you in the wrong file.
2. **Two of the bugs interact.** Each one individually would produce loud, obviously-wrong output. Together they cancel on the most-visible output lines. The only signal that exposes them is a *secondary* consistency check, not a direct disagreement with the expected output.
3. **One bug is hidden behind another.** Its symptom is masked by a louder bug. After you fix the louder bug, the hidden one's symptom will appear — possibly on a line that previously looked "explainable." If a line that you thought was caused by a bug you've already located is *still* wrong after your fix, do not assume you fixed it incorrectly: look for an additional bug.
4. **Fixing one bug in isolation can make the visible output look worse.** If you fix a real bug and now the output disagrees with the expected output *more* than before, do not assume you broke something — you may have just removed a piece of accidental compensation.

You may not run the program until you have submitted your answer. Trace by reading.

Expected output

```
Tokens (41 total):
  ["the","cat","sat","on","the","mat","a","high","energy","dog","saw","
     the",
   "cat","the","cat","ran","from","the","long","haired","dog","the","
      dog",
   "was","fast","but","the","cat","was","faster","anna","saw","the","
      high",
   "end","racecar","and","the","upper","level","rotator"]

Word frequencies:
  the: 9
  cat: 4
  sat: 1
  on: 1
  mat: 1
  a: 1
  high: 2
  energy: 1
  dog: 3
  saw: 2
  ran: 1
  from: 1
  long: 1
  haired: 1
```

```
  was: 2
  fast: 1
  but: 1
  faster: 1
  anna: 1
  end: 1
  racecar: 1
  and: 1
  upper: 1
  level: 1
  rotator: 1
Total token count (via freq table): 41

Longest word: racecar

Palindromes: ["anna","racecar","level","rotator"]

Top 3 most frequent:
  the: 9
  cat: 4
  dog: 3

3 least frequent:
  sat: 1
  on: 1
  mat: 1
```

Notes on the expected output:

- `Word frequencies` is in first-seen order — the order each word first appears in the tokenized stream.
- `Total token count (via freq table)` is a self-consistency check: the sum of all counts in the frequency table should equal the number of tokens. The driver computes it directly by summing the frequency table; if it disagrees with the `Tokens (N total)` header, something inside `word_freq` is producing the wrong internal representation.
- `3 least frequent` shows three of the many singleton words. Because every singleton has count 1, sort stability decides the order: `lists:sort` is stable, so the three earliest singletons by first-seen order win.

Your write-up

For each bug, record:

1. **Location** — file name and line number(s).
2. **Diagnosis** — in 1–2 sentences, what is wrong.
3. **Fix** — the corrected line(s).
4. **Symptom** — which line(s) of the expected output differ from what the buggy program actually prints, and why this specific bug causes that specific symptom. If a bug has no *direct* visible symptom (because another bug masks it), explain how you knew it was there anyway.